

Controle de Fluxo: Tomada de Decisão

Condições

O programa avalia expressões lógicas para decidir qual caminho seguir.

Estrutura if/else

Permite executar blocos de código diferentes conforme a condição seja verdadeira ou falsa.

Adaptação

O código se adapta a situações diferentes, tornando o programa mais inteligente e flexível.

Exemplo

```
x = 10
if x > 5:
    print("Maior que 5")
else:
    print("5 ou menos")
```

Saída

Maior que 5

Trabalhando com Dados Simples

Valores individuais

Podemos representar uma única pessoa usando variáveis escalares. Cada variável guarda um valor único na memória.

```
weight = 60
height = 1.75
subject = "A"
healthy = True
bmi = weight / height**2
bmi
```

Cálculo de IMC

Vamos calcular o IMC de um indivíduo usando valores simples.

 Resultado: 19.591836734693878

Evoluindo para Múltiplos Valores

Usando vetores (arrays)

Quando temos várias pessoas, usamos arrays do NumPy para guardar múltiplos valores. Para processá-los, podemos usar estruturas de repetição ou, como veremos no final da aula, operações vetorizadas.

```
import numpy as np

weight = np.array([60, 72, 57, 90, 95, 72])
height = np.array([1.75, 1.80, 1.65, 1.90, 1.74, 1.91])
subject = np.array(["A", "B", "C", "D", "E", "F"])

weight
```

Saída

```
array([60, 72, 57, 90, 95, 72])
```

Múltiplas Condições (if / elif / else)

Várias alternativas

elif permite testar uma nova condição quando a anterior é falsa.

Ordem de avaliação

As condições são testadas de cima para baixo; a primeira verdadeira é executada.

Caso padrão

O else trata todos os casos restantes.

```
nota = 7
if nota >= 9:
    conceito = "A"
elif nota >= 7:
    conceito = "B"
elif nota >= 6:
    conceito = "C"
else:
    conceito = "D"
conceito # 'B'
```

Exemplo de Decisão com if

Condição avaliada

O if avalia uma única condição lógica:

`nota >= 6`

Decisão tomada

O fluxo do programa depende do valor atual da variável.

Resultado final

Se verdadeira, um bloco executa; caso contrário, o else executa.

```
nota = 7
if nota >= 6:
    resultado = "aprovado"
else:
    resultado = "reprovado"
resultado
```

📄 Saída: 'aprovado'

Condições em Vetores: if vs condicional vetorizada

if tradicional

Espera um único valor booleano (True ou False). Não funciona diretamente com vetores.

np.where

Para aplicar uma condição a cada elemento de um vetor, usamos operações vetorizadas no NumPy.

```
import numpy as np
```

```
alturas = np.array([1.65, 1.80, 1.55])
```

```
classe = np.where(alturas < 1.70, "baixa", "alta")
```

```
classe
```

Saída: array(['baixa', 'alta', 'baixa'], dtype='<U5')

Condicional Vetorizada com np.where

Elemento a elemento

A condição é avaliada para cada posição do vetor

Novo vetor

O resultado é um novo vetor com os valores escolhidos

Transformação em lote

Permite transformar dados de forma eficiente

```
alturas = np.array([1.65, 1.80, 1.55])
classe = np.where(alturas < 1.70, "baixa", "alta")
classe
```

Altura	Condição	Classe
1.65	< 1.70 ✓	baixa
1.80	< 1.70 ✕	alta
1.55	< 1.70 ✓	baixa

Repetição: Quando e Por Que Usar

Quando repetir

Repetição aplica a mesma operação várias vezes de forma automática.

Laços (for, while)

Controlam a execução passo a passo, iterando sobre elementos.

Operações vetorizadas

São mais rápidas quando possíveis, processando todos os dados de uma vez.

```
import numpy as np

x = np.array([1, 2, 3, 4])
y = x**2
y
```

Saída: array([1, 4, 9, 16])

Relação entre Condição e Repetição

Condições decidem

Determinam se algo deve ou não executar

Repetições executam

Realizam operações várias vezes

Combinação poderosa

Condições dentro do loop controlam quando agir

```
i = 1
while i <= 5:
    if i % 2 == 0:
        print(i, "é par")
    i += 1
```

📄 Saída: 2 é par e 4 é par

Estrutura de Repetição: for

O que é o for

Usamos **for** quando sabemos quantas vezes o código deve ser executado. Ele percorre sequências como listas, arrays ou intervalos.

Iteração sobre sequências

Número definido de repetições com sintaxe limpa e legível.

Saída

Imprime os números de 1 a 5, um por linha.

```
for i in range(1, 6):  
    print(i)
```

Calculando IMC com for (forma idiomática)

Percorrendo vetores em paralelo

A função `zip` permite iterar sobre dois vetores ao mesmo tempo.

Evitando índices manuais

Não precisamos gerenciar índices explicitamente.

Código mais seguro

O código fica mais simples, limpo e menos propenso a erros.

```
import numpy as np

weight = np.array([60, 72, 57, 90, 95, 72])
height = np.array([1.75, 1.80, 1.65, 1.90, 1.74, 1.91])

bmi = []
for w, h in zip(weight, height):
    bmi.append(w / h**2)

bmi = np.array(bmi)
bmi
```

📄 Resultado: `array([19.59, 22.22, 20.94, 24.93, 31.38, 19.74])`

Inspecionando os Cálculos

Acompanhando o loop

A função `print()` dentro do loop mostra cada passo da execução, revelando a lógica interna do programa.

Saída progressiva

O vetor começa com zeros e cada posição é calculada uma por vez.

```
import numpy as np

bmi = np.zeros(len(weight))
for i in range(len(weight)):
    bmi[i] = weight[i] / height[i]**2
print(bmi)
```

Depurando no VS Code

Breakpoints

Permitem parar o programa em pontos específicos para análise detalhada.

Observando variáveis

Podemos ver os valores de todas as variáveis em tempo real a cada iteração.

Facilitando correções

Facilita o entendimento de erros e da lógica do programa.

```
for w, h in zip(weight, height):  
    bmi_val = w / h**2 # breakpoint aqui  
    print(w, h, bmi_val)
```

Cada iteração mostra os valores de peso, altura e IMC calculado, permitindo verificar a correção dos cálculos.

Escopo, Nomes e Reprodutibilidade

Escopo de variáveis

Variáveis criadas dentro de uma função são locais a ela. Blocos if, for e while não criam escopo novo: o que é definido dentro deles continua existindo depois.

Reprodutibilidade

Rodar o script do início ao fim garante resultados consistentes e previsíveis.

Evitando erros

Evita resultados errados causados por resíduos de memória de execuções anteriores.

```
def compute():  
    x = 10      # x é local à função  
    return x * 2  
  
compute()      # 20  
# print(x)     # NameError: x não existe fora da função  
  
for i in range(3):  
    pass  
print(i)       # 2: i continua existindo após o for
```

Estrutura de Repetição: while

O que é o while

Executa o código enquanto uma condição for verdadeira.

Repetições indefinidas

O número de repetições não é conhecido previamente.

Cuidado essencial

A condição deve mudar para evitar loops infinitos.

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Calculando IMC com while

Inicialização

Precisamos inicializar a variável de controle antes do loop.

Condição

A condição decide se o loop continua ou para.

Atualização

O índice é atualizado manualmente a cada iteração.

```
import numpy as np

i = 0
bmi = np.zeros(len(weight))
while i < len(weight):
    bmi[i] = weight[i] / height[i]**2
    i += 1
```

```
bmi
```

📄 **Saída:** array([19.59, 22.22, 20.94, 24.93, 31.38, 19.74])

Encapsulando o Cálculo em uma Função

Organização e reutilização

Funções agrupam código que realiza uma tarefa específica, tornando o programa mais organizado e reutilizável.

Cálculo com while

Vamos calcular o IMC usando uma função com **while**.

```
import numpy as np

def compute_bmi(weight, height):
    i = 0
    bmi = np.zeros(len(weight))
    while i < len(weight):
        bmi[i] = weight[i] / height[i]**2
        i += 1
    return bmi

bmi = compute_bmi(weight, height)
bmi
```

Resultado: array([19.59, 22.22, 20.94, 24.93, 31.38, 19.74])

Implementando do Jeito Mais Eficiente

Sem loops necessários

Não precisamos de loops para este cálculo específico.

Operação vetorizada

O NumPy opera em vetores inteiros de uma só vez.

Código otimizado

O código fica mais rápido, limpo e pythônico.

```
def compute_bmi(weight, height):  
    return weight / height**2
```

```
bmi = compute_bmi(weight, height)  
bmi
```

 **Saída:** array([19.59, 22.22, 20.94, 24.93, 31.38, 19.74])

Usando a Função com Escalares e Vetores

Função única

Uma mesma implementação serve para diferentes tipos de dados.

Aplicação automática

O NumPy aplica automaticamente a operação ao tipo correto.

Loops desnecessários

Evitamos escrever loops manuais, simplificando o código.

Com escalar (um valor)

```
compute_bmi(80, 1.79)
```

Saída: 24.97

Com vetores (múltiplos valores)

```
compute_bmi(weight, height)
```

Saída: array([19.59, 22.22, 20.94, 24.93, 31.38, 19.74])

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Introdução essencial aos fundamentos de programação e pensamento computacional com Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Guia completo das ferramentas essenciais para trabalhar com dados em Python, incluindo NumPy, Pandas e visualização.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.

Aprenda os fundamentos de ciência de dados construindo algoritmos e ferramentas do zero com Python.